Strings in Java

Freshman Level: First Steps with Text

Haluk O. Bingol

Faculty of Computer and Information Sciences
Yeditepe University

v2026-10-14

Outline

1. Running Java on Your Computer
2. Your First Strings
3. Reading Text
4. Character by Character
5. Comparing Strings
6. When Things Go Wrong
7. Small Problems
8. Summary

Running Java on Your Computer

Install Java and check it

You need a **JDK** (Java Development Kit), for example Eclipse Temurin 21 from `adoptium.net`.

Windows

1. Download the `.msi` installer, run it, and tick *Set JAVA_HOME* and *Add to PATH*
2. Open **Command Prompt** (`Win` key, type `cmd`)
3. Check:

```
java -version
javac -version
```

macOS

1. Download the `.pkg` installer (Apple silicon: *aarch64*; Intel: *x64*) and run it
2. Open **Terminal** (`Cmd`+`Space`, type `Terminal`)
3. Check:

```
java -version
javac -version
```

Both commands should print a version number such as `21.0.10`.

Compile and run a program

Windows (Command Prompt)

```
cd C:\Users\ayse\cse101
chcp 65001
javac -encoding UTF-8 Greeting.java
java Greeting
```

`chcp 65001` lets the console show Turkish letters (ç, ğ, ı, ş).

macOS (Terminal)

```
cd ~/cse101
javac Greeting.java
java Greeting
```

Terminal is UTF-8 already; no extra step.

- `javac` **compiles** `Greeting.java` into `Greeting.class`; `java` **runs** it (no `.class` at the end)
- Shortcut since Java 11 for one-file programs: `java Greeting.java`
- The file name must match the class name: `public class Greeting` lives in `Greeting.java`

Your First Strings

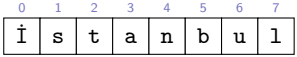
What is a string?

A **string** is a piece of text: a sequence of characters between double quotes.

```
String city = "İstanbul";
String empty = "";
char letter = 'A';      // one char
```

- `String` (capital S) is the type
- Double quotes `"..."` for strings, single quotes `'.'` for one `char`
- `+` glues strings together (**concatenation**)

Every character has a position, starting at **0**:



`city.length()` is 8,
`city.charAt(0)` is `'İ'`,
`city.charAt(7)` is `'l'`.

Joining strings with +

codeoutput/Greeting.java **Java**

```java
public class Greeting {
    public static void main(String[] args) {
        String first = "Ayşe";
        String last = "Yılmaz";
        String full = first + " " + last;
        System.out.println("Hello, " + full + "!");
        System.out.println(full + " has " + full.length() + " chars"
    );
        int year = 2026;
        System.out.println("Class of " + (year + 4));
        System.out.println("Class of " + year + 4);
    }
}
```

Look at the last line: `"Class of " + year` is a string, then `+ 4` appends “4”. Use parentheses to add numbers first.

Joining strings with +

codeoutput/Greeting.java — Java

```java
public class Greeting {
    public static void main(String[] args) {
        String first = "Ayşe";
        String last = "Yılmaz";
        String full = first + " " + last;
        System.out.println("Hello, " + full + "!");
        System.out.println(full + " has " + full.length() + " chars"
    );
        int year = 2026;
        System.out.println("Class of " + (year + 4));
        System.out.println("Class of " + year + 4);
    }
}
```

Output
codeoutput/Greeting.txt

```
Hello, Ayşe Yılmaz!
Ayşe Yılmaz has 11 chars
Class of 2030
Class of 20264
```

Look at the last line: `"Class of " + year` is a string, then `+ 4` appends “4”. Use parentheses to add numbers first.

Puzzle: what does + do here?

codeoutput/PlusPuzzle.java **Java**

```java
public class PlusPuzzle {
    public static void main(String[] args) {
        System.out.println(1 + 2 + "3");
        System.out.println("1" + 2 + 3);
        System.out.println("1" + (2 + 3));
        System.out.println('a' + 'b');
        System.out.println("" + 'a' + 'b');
        System.out.println('a' + 1);
        System.out.println((char) ('a' + 1));
    }
}
```

+ works **left to right**. Number + number adds; as soon as one side is a `String`, it joins. Two `chars` are numbers: `'a' + 'b'` is 97 + 98.

Puzzle: what does + do here?

codeoutput/PlusPuzzle.java — Java

```java
public class PlusPuzzle {
    public static void main(String[] args) {
        System.out.println(1 + 2 + "3");
        System.out.println("1" + 2 + 3);
        System.out.println("1" + (2 + 3));
        System.out.println('a' + 'b');
        System.out.println("" + 'a' + 'b');
        System.out.println('a' + 1);
        System.out.println((char) ('a' + 1));
    }
}
```

Output
codeoutput/PlusPuzzle.txt

```
33
123
15
195
ab
98
b
```

+ works **left to right**. Number + number adds; as soon as one side is a `String`, it joins. Two `chars` are numbers: `'a' + 'b'` is 97 + 98.

Reading Text

Reading a line with Scanner

codeoutput/ReadName.java — Java

```java
import java.util.Scanner;

public class ReadName {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        System.out.print("Your name: ");
        String name = in.nextLine();
        System.out.print("Your city: ");
        String city = in.nextLine();
        System.out.println();
        System.out.println("Hi " + name + " from " + city + "!");
        System.out.println("Your name starts with " + name.charAt(0)
    );
    }
}
```

Here the input comes from the file `ReadName.in` (`Ayşe Yılmaz`, `İstanbul`), so the typed text is not shown after the prompts. `nextLine()` reads a whole line, spaces included.

Reading a line with Scanner

codeoutput/ReadName.java — Java

```java
import java.util.Scanner;

public class ReadName {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        System.out.print("Your name: ");
        String name = in.nextLine();
        System.out.print("Your city: ");
        String city = in.nextLine();
        System.out.println();
        System.out.println("Hi " + name + " from " + city + "!");
        System.out.println("Your name starts with " + name.charAt(0)
    );
    }
}
```

Output
codeoutput/ReadName.txt

```
Your name: Your city:
Hi Ayşe Yılmaz from İ
     stanbul!
Your name starts with A
```

Here the input comes from the file `ReadName.in` (`Ayşe Yılmaz`, `İstanbul`), so the typed text is not shown after the prompts. `nextLine()` reads a whole line, spaces included.

The nextInt / nextLine trap

codeoutput/NextLineTrap.java **Java**

```java
import java.util.Scanner;

public class NextLineTrap {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        System.out.print("Age: ");
        int age = in.nextInt();          // reads 19, not the Enter
        System.out.print("Name: ");
        String name = in.nextLine();     // reads the rest: ""
        System.out.println();
        System.out.println("[" + name + "] is " + age);
        name = in.nextLine();            // now the real name
        System.out.println("[" + name + "] is " + age);
    }
}
```

Input: 19, Enter, `Ali Can`, Enter. `nextInt()` stops before the Enter, so the first `nextLine()` returns the empty rest of that line. Fix: call `in.nextLine()` once after `nextInt()`.

The nextInt / nextLine trap

codeoutput/NextLineTrap.java — Java

```java
import java.util.Scanner;

public class NextLineTrap {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        System.out.print("Age: ");
        int age = in.nextInt();          // reads 19, not the Enter
        System.out.print("Name: ");
        String name = in.nextLine();     // reads the rest: ""
        System.out.println();
        System.out.println("[" + name + "] is " + age);
        name = in.nextLine();            // now the real name
        System.out.println("[" + name + "] is " + age);
    }
}
```

Output
codeoutput/NextLineTrap.txt

```
Age: Name:
[] is 19
[Ali Can] is 19
```

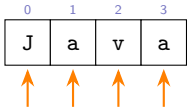
Input: 19, Enter, `Ali Can`, Enter. `nextInt()` stops before the Enter, so the first `nextLine()` returns the empty rest of that line. Fix: call `in.nextLine()` once after `nextInt()`.

Character by Character

Looping over a string

```
for (int i = 0; i < s.length(); i++) {
    char c = s.charAt(i);
    // use c
}
```

- Start at 0
- Stop **before** `s.length()`: use <, not <=
- `charAt(i)` gives the `char` at position `i`



`i` = 4 = `length()`: stop

Spelling a word

codeoutput/Spell.java **Java**

```java
public class Spell {
    public static void main(String[] args) {
        String word = "Java";
        for (int i = 0; i < word.length(); i++) {
            char c = word.charAt(i);
            System.out.println("index " + i + ": " + c);
        }
        for (char c : word.toCharArray()) {   // for-each
            System.out.print(c + "-");
        }
        System.out.println();
    }
}
```

Spelling a word

codeoutput/Spell.java **Java**

```java
public class Spell {
    public static void main(String[] args) {
        String word = "Java";
        for (int i = 0; i < word.length(); i++) {
            char c = word.charAt(i);
            System.out.println("index " + i + ": " + c);
        }
        for (char c : word.toCharArray()) {   // for-each
            System.out.print(c + "-");
        }
        System.out.println();
    }
}
```

Output
codeoutput/Spell.txt

```
index 0: J
index 1: a
index 2: v
index 3: a
J-a-v-a-
```

Counting vowels

codeoutput/CountVowels.java **Java**

```java
public class CountVowels {
    public static void main(String[] args) {
        String text = "Bilgisayar Mühendisliği";
        String vowels = "aeıioöuüAEIİOÖUÜ";
        int count = 0;
        for (int i = 0; i < text.length(); i++) {
            if (vowels.indexOf(text.charAt(i)) >= 0) {
                count++;
            }
        }
        System.out.println(text);
        System.out.println("vowels: " + count);
    }
}
```

A handy trick: `vowels.indexOf(c) >= 0` means “c is one of the characters in `vowels`”.

Counting vowels

codeoutput/CountVowels.java — Java

```java
public class CountVowels {
    public static void main(String[] args) {
        String text = "Bilgisayar Mühendisliği";
        String vowels = "aeıioöuüAEIİOÖUÜ";
        int count = 0;
        for (int i = 0; i < text.length(); i++) {
            if (vowels.indexOf(text.charAt(i)) >= 0) {
                count++;
            }
        }
        System.out.println(text);
        System.out.println("vowels: " + count);
    }
}
```

Output
codeoutput/CountVowels.txt

```
Bilgisayar Mühendisliği
vowels: 9
```

A handy trick: `vowels.indexOf(c) >= 0` means “c is one of the characters in `vowels`”.

Comparing Strings

The most famous string bug

codeoutput/EqualsTrap.java **Java**

```java
import java.util.Scanner;

public class EqualsTrap {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        String answer = in.nextLine();          // user types: yes
        System.out.println(answer == "yes");    // same object?
        System.out.println(answer.equals("yes"));
        String lit = "yes";
        System.out.println(lit == "yes");       // both literals
    }
}
```

The user typed `yes`, yet `answer == "yes"` is `false`: `==` asks whether they are the **same object**.

Always compare text with `equals`: `answer.equals("yes")`.

The most famous string bug

codeoutput/EqualsTrap.java — Java

```java
import java.util.Scanner;

public class EqualsTrap {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        String answer = in.nextLine();          // user types: yes
        System.out.println(answer == "yes");    // same object?
        System.out.println(answer.equals("yes"));
        String lit = "yes";
        System.out.println(lit == "yes");       // both literals
    }
}
```

Output
codeoutput/EqualsTrap.txt

```
false
true
true
```

The user typed `yes`, yet `answer == "yes"` is `false`: `==` asks whether they are the **same object**.
Always compare text with `equals`: `answer.equals("yes")`.

Comparing strings: the rules

Question	Write
same text?	`a.equals(b)`
same text, ignoring case?	`a.equalsIgnoreCase(b)`
which comes first?	`a.compareTo(b) < 0`
is it empty?	`a.isEmpty()`
does it contain `"x"`?	`a.contains("x")`
does it start with `"Dr."`?	`a.startsWith("Dr.")`

Never

`if (a == b)` with strings. It may even work in a small test (with literals), then fail with user input.

When Things Go Wrong

Index out of bounds

codeoutput/OutOfBounds.java — Java

```java
public class OutOfBounds {
    public static void main(String[] args) {
        String s = "Java";
        System.out.println(s.charAt(3));
        System.out.println(s.charAt(4));    // no index 4
    }
}
```

Read the message: the exception name, `Index 4 out of bounds for length 4`, and the line of *your* file (`OutOfBounds.java:5`). Valid indices are `0..length()-1`. (The 10 lines from inside Java, `at java.base/...`, are skipped here.)

Index out of bounds

codeoutput/OutOfBounds.java — Java

```java
public class OutOfBounds {
    public static void main(String[] args) {
        String s = "Java";
        System.out.println(s.charAt(3));
        System.out.println(s.charAt(4));    // no index 4
    }
}
```

Output — codeoutput/OutOfBounds.txt

```
a
Exception in thread "main" java.lang.StringIndexOutOfBoundsException: Index 4 out of bounds for length
      4
        at OutOfBounds.main(OutOfBounds.java:5)
```

Read the message: the exception name, `Index 4 out of bounds for length 4`, and the line of *your* file (`OutOfBounds.java:5`). Valid indices are `0..length()-1`. (The 10 lines from inside Java, `at java.base/...`, are skipped here.)

A string that is not there: `null`

codeoutput/NullName.java — Java

```java
public class NullName {
    static String name;                    // not set: null

    public static void main(String[] args) {
        System.out.println("name is " + name);
        System.out.println(name.length());
    }
}
```

`name` refers to no object. Printing it shows `null`, but calling a method on it crashes. Java even tells us which variable was `null`.

A string that is not there: `null`

codeoutput/NullName.java — Java

```java
public class NullName {
    static String name;                     // not set: null

    public static void main(String[] args) {
        System.out.println("name is " + name);
        System.out.println(name.length());
    }
}
```

Output — codeoutput/NullName.txt

```
name is null
Exception in thread "main" java.lang.NullPointerException: Cannot invoke "String.length()" because "
     NullName.name" is null
        at NullName.main(NullName.java:6)
```

`name` refers to no object. Printing it shows `null`, but calling a method on it crashes. Java even tells us which variable was `null`.

Small Problems

Reversing a string

codeoutput/Reverse.java — Java

```java
public class Reverse {
    public static void main(String[] args) {
        String s = "Yeditepe";
        String r = "";
        for (int i = s.length() - 1; i >= 0; i--) {
            r = r + s.charAt(i);
        }
        System.out.println(r);
        System.out.println(new StringBuilder(s).reverse());
    }
}
```

The loop walks from the last index down to 0. `StringBuilder` does the same in one call, and faster.

Reversing a string

codeoutput/Reverse.java — Java

```java
public class Reverse {
    public static void main(String[] args) {
        String s = "Yeditepe";
        String r = "";
        for (int i = s.length() - 1; i >= 0; i--) {
            r = r + s.charAt(i);
        }
        System.out.println(r);
        System.out.println(new StringBuilder(s).reverse());
    }
}
```

Output
codeoutput/Reverse.txt

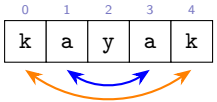
```
epetideY
epetideY
```

The loop walks from the last index down to 0. `StringBuilder` does the same in one call, and faster.

Palindrome: the idea

A **palindrome** reads the same backwards: `kayak`, `level`, `ey edip adanada pide ye`.

Compare the two ends, then move inwards:



`i` moves right, `j` moves left

Algorithm 1: IsPalindrome(s)

```
1 i ← 0,    j ← length(s) − 1;
2 while i < j do
3     if s[i] ≠ s[j] then
4         return false;
5     i ← i + 1,    j ← j − 1;
6 return true;
```

Palindrome in Java

codeoutput/Palindrome.java — Java

```java
public class Palindrome {
    static boolean isPalindrome(String s) {
        s = s.toLowerCase();
        int i = 0, j = s.length() - 1;
        while (i < j) {
            if (s.charAt(i) != s.charAt(j)) {
                return false;
            }
            i++;
            j--;
        }
        return true;
    }

    public static void main(String[] args) {
        String[] words = {"kayak", "Level", "java",
                          "ey edip adanada pide ye"};
        for (String w : words) {
            System.out.println(w + " -> " + isPalindrome(w));
        }
    }
}
```

`toLowerCase()` first, so `"Level"` counts. The spaces in the last example happen to be symmetric too.

Palindrome in Java

codeoutput/Palindrome.java — Java

```java
public class Palindrome {
    static boolean isPalindrome(String s) {
        s = s.toLowerCase();
        int i = 0, j = s.length() - 1;
        while (i < j) {
            if (s.charAt(i) != s.charAt(j)) {
                return false;
            }
            i++;
            j--;
        }
        return true;
    }

    public static void main(String[] args) {
        String[] words = {"kayak", "Level", "java",
                          "ey edip adanada pide ye"};
        for (String w : words) {
            System.out.println(w + " -> " + isPalindrome(w));
        }
    }
}
```

Output
codeoutput/Palindrome.txt

```
kayak -> true
Level -> true
java -> false
ey edip adanada pide ye
      -> true
```

`toLowerCase()` first, so `"Level"` counts. The spaces in the last example happen to be symmetric too.

Counting lines, words and characters

codeoutput/WordCount.java — Java

```java
import java.util.Scanner;

public class WordCount {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        int lines = 0, words = 0, chars = 0;
        while (in.hasNextLine()) {
            String line = in.nextLine();
            lines++;
            chars += line.length();
            if (!line.isBlank()) {
                words += line.strip().split("\\s+").length;
            }
        }
        System.out.println(lines + " lines, " + words + " words, "
                + chars + " chars");
    }
}
```

Input: 3 lines, one empty. `split("\\s+")` splits at runs of spaces; `strip()` avoids an empty first word.

Counting lines, words and characters

codeoutput/WordCount.java — Java

```java
import java.util.Scanner;

public class WordCount {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        int lines = 0, words = 0, chars = 0;
        while (in.hasNextLine()) {
            String line = in.nextLine();
            lines++;
            chars += line.length();
            if (!line.isBlank()) {
                words += line.strip().split("\\s+").length;
            }
        }
        System.out.println(lines + " lines, " + words + " words, "
                + chars + " chars");
    }
}
```

Output
codeoutput/WordCount.txt

```
3 lines, 7 words, 52 chars
```

Input: 3 lines, one empty. `split("\\s+")` splits at runs of spaces; `strip()` avoids an empty first word.

Summary

Do and do not

Do

- Count positions from **0** to `length()-1`
- Loop with `i < s.length()`
- Compare with `equals`
- Store results: `s = s.trim();`
- Put numbers in parentheses: `"x" + (a + b)`
- Call `nextLine()` after `nextInt()`

Do not

- Compare strings with `==`
- Use `charAt(s.length())`
- Call methods on a `null` string
- Confuse `'a'` (char) with `"a"` (String)
- Expect `s.toUpperCase();` alone to change `s`

Exercises

1. Read a full name and print the initials: `Ayşe Yılmaz` → `A.Y.`
2. Read a word and print it with every letter on its own line, numbered from 1.
3. Count how many times the letter `a` (either case) appears in a sentence.
4. Read a password and say whether it has at least 8 characters, a digit and an upper-case letter (use `Character.isDigit`, `Character.isUpperCase`).
5. Read two words and print them in dictionary order (`compareToIgnoreCase`).
6. Replace every space in a sentence with `_` **without** `replace`: build a new string in a loop.
7. **Challenge:** check whether a sentence is a palindrome when spaces are ignored (`"Ey Edip, Adana'da pide ye"`).

Questions?